

IT3012 – Lecture 04

Searching & Problem Solving in AI

Jeewaka Perera

From Agents to Search

- **Last Weeks:**

- Defined *intelligent agents* and *rationality*.
- Modeled task environments using the **PEAS** framework.
- Explored the 5 core **Agent Architectures**, learning that agents must choose actions to maximize performance.

- **The Challenge:**

- *Reflex Agents* only react to the present (IF-THEN rules) and easily get trapped in complex environments.
- *Goal-Based Agents* overcome this by asking: "*What will happen if I do action A? Will it reach my goal?*". But **how** do they actually compute the answer to that question?

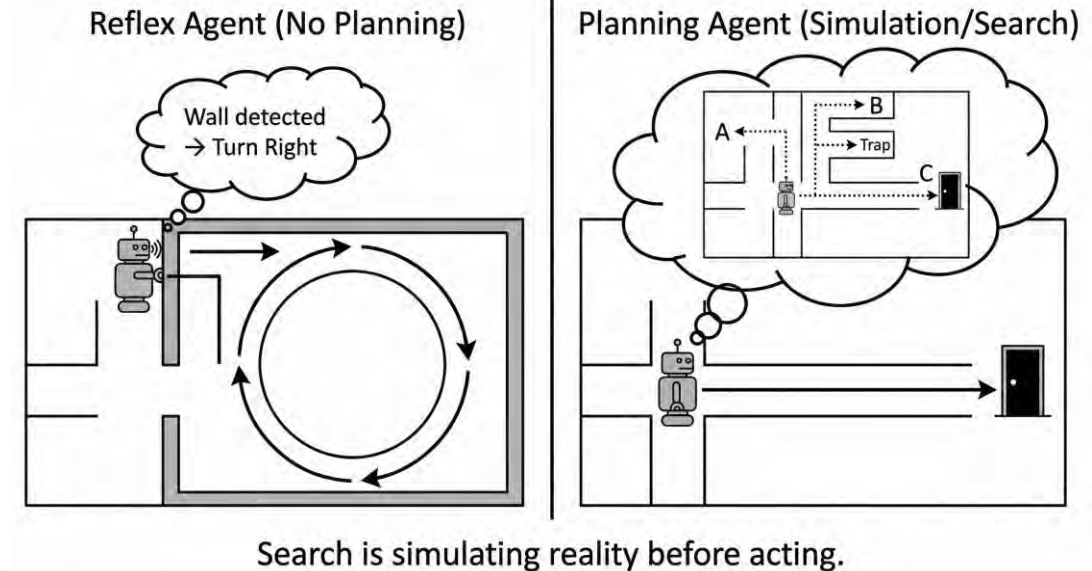
From Agents to Search

- **This Week:**

- Learn **Uninformed Search**: The offline simulation engine that powers Goal-Based Agents.
- Connect **problem formulation** → **search** → **solution**.
- See how search algorithms (BFS, DFS, UCS) become the **foundation** for path planning.

Simple Reflex vs Goal Based

- **The Problem with Reflexes:** Simple Reflex Agent gets trapped in infinite loops in partially observable environments.
- **The Search Solution:** Goal-Based Agents simulate every possible future move until they find a "Win" state.
- Search is essentially "offline simulation." The agent explores the map in its "mind" before sending a command to its actuators.



Why Search is the Backbone of AI Decision-Making

- **Search = Systematic exploration** of possible actions to reach a goal.
- **Without search**, many agents can't choose optimal actions in unfamiliar environments.
- Powers many AI applications:
 - **Navigation:** Google Maps finding shortest routes.
 - **Games:** Chess AI exploring possible moves.
 - **Robotics:** Path planning for warehouse robots.
 - **Problem Solving:** Solving puzzles like the 8-puzzle or Sudoku.
- **Core idea:** *From problem formulation → search → solution.*

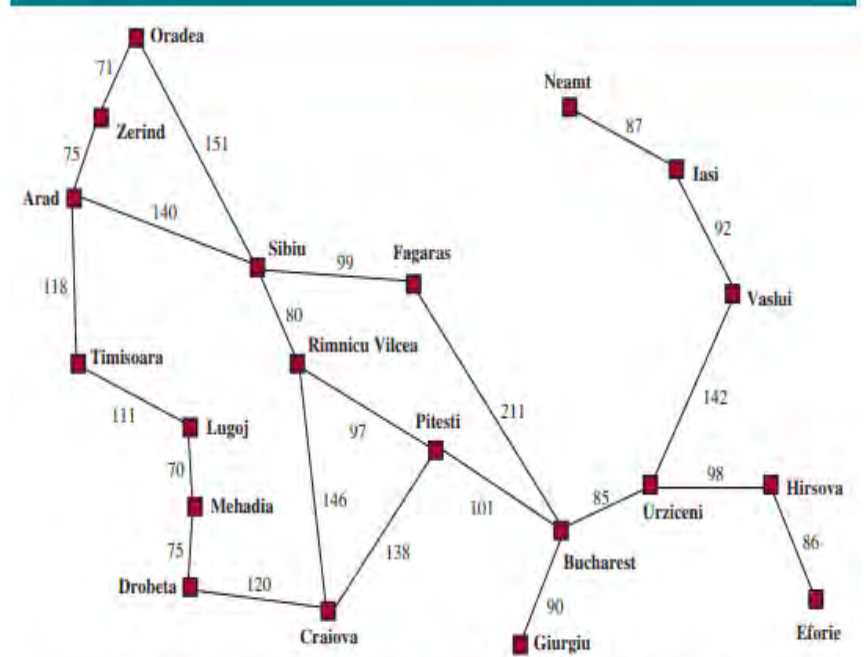
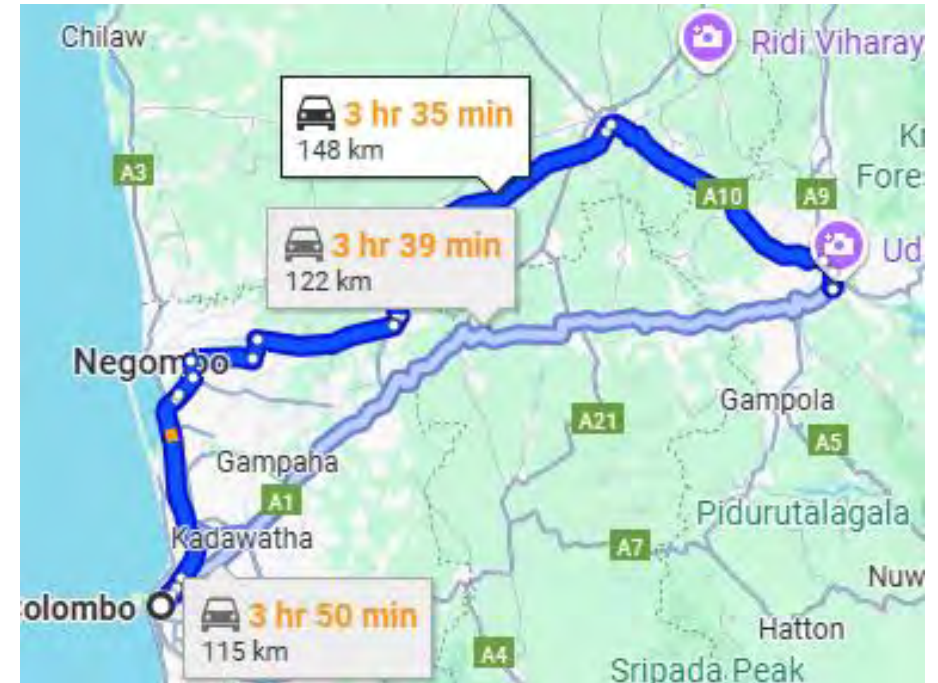


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Why Search is the Backbone of AI Decision-Making

- **Problem Formulation**
 - Starting point: Colombo; Goal: Kandy.
- **Search Process**
 - Explore different possible routes via highway vs local roads.
- **Solution Found**
 - Select shortest travel time route



NOTE : SEARCH HAPPENS INSIDE THE AGENT BEFORE CHOOSING THE ACTION
The driver doesn't move until Google Maps gives a route.

In class activity

Campus Delivery Robot

- Deliver packages to 3 buildings starting from the main gate.
- Battery lasts 30 minutes.

Emergency Evacuation

- Evacuate people from a building with blocked exits.
- Must find shortest safe route to the assembly point.

Library Book Search

- Find a specific book in a large library.
- Some sections under maintenance (inaccessible).

Warehouse Order Picking

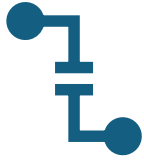
- Pick 5 items from different shelves in minimal walking distance.
- Heavy items must be collected last.

Fill in the 3 stages

- **Problem Formulation** (*start, goal, constraints*)
- **Search Process** (*options to explore*)
- **Solution Found** (*final decision*)



Formalizing a Search Problem



Initial State – where the agent is.

This is the agent's current percept sequence or "What the world is like now"

Example: Campus gate in delivery robot scenario.



Actions – possible moves or operations.

Example: Move to building A, move to building B, recharge battery.



Transition Model – what each action does.

This is the exact same "How the world evolves" and "What my actions do" block from the Goal-Based Agent diagram

Example: Moving to building A takes 5 minutes and reduces battery by 10%.



Goal Test – determines if we've succeeded.

This maps to the "Goals" block that the agent checks against.

Example: All packages delivered.

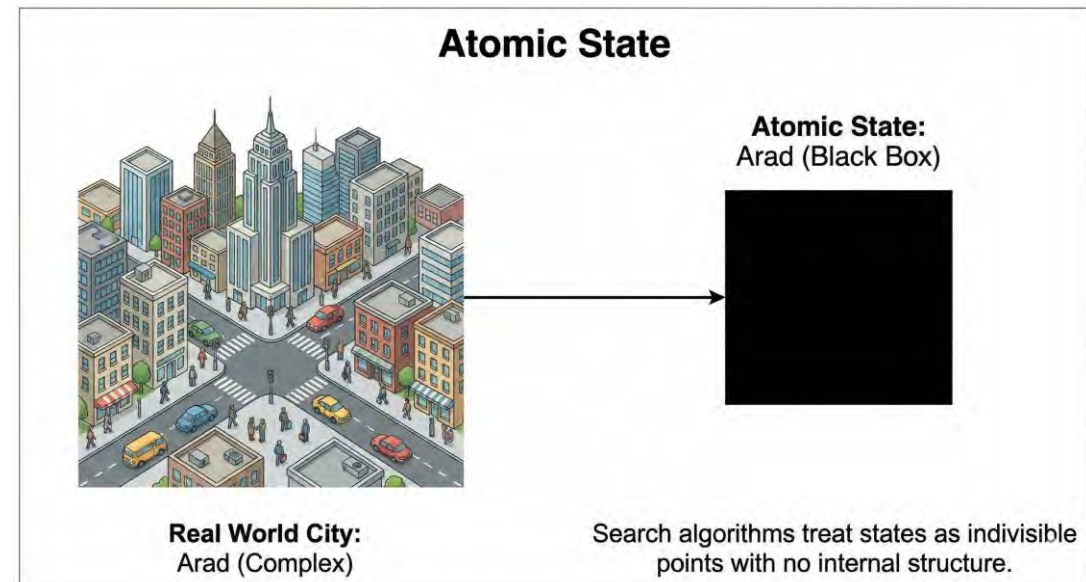


Path Cost Function –
numeric cost of a path.

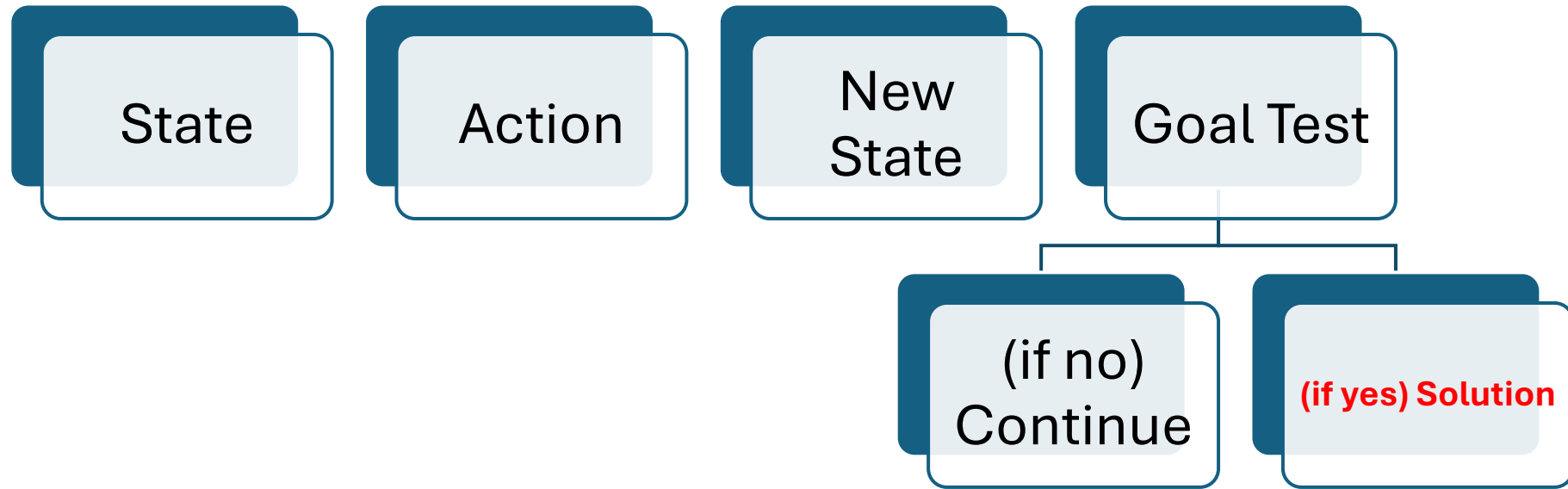
Example: Time taken or battery used.

A Critical Definition: Atomic Representations

- In Classical Search algorithms (like A* or BFS), states are **Atomic**.
- **"Atomic" means Indivisible:**
 - The algorithm treats each state (e.g., "Arad") as a **Black Box**.
 - It has **no internal structure** (no visible attributes like traffic, fuel, or population).
 - The agent only asks one question: *"Does this state label match the Goal state label?"*



Formalizing a Search Problem



- The *search problem definition* is how we describe the decision-making challenge inside the agent before it acts.
- Choosing the right state representation and cost function changes the whole search outcome.

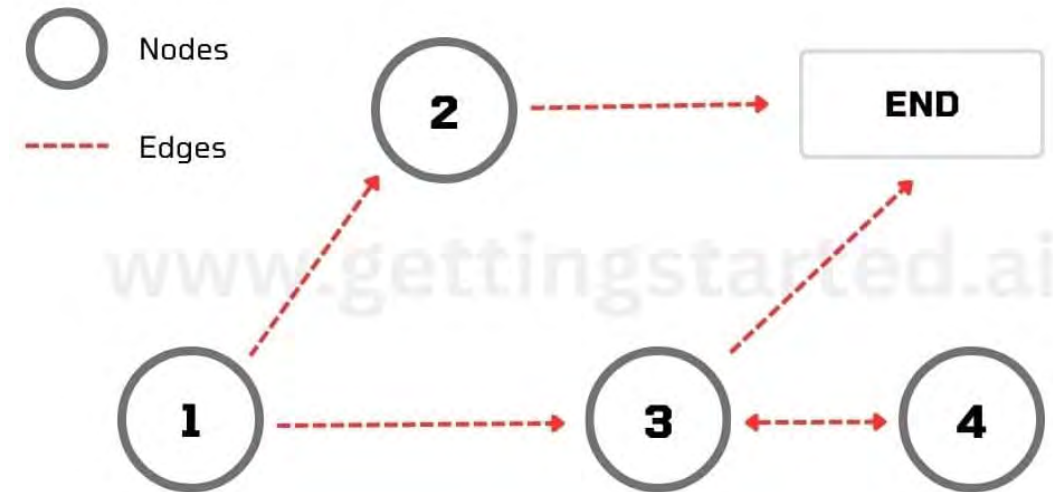
State Space Representation

Definition:

- The set of all possible states reachable from the initial state, connected by actions.

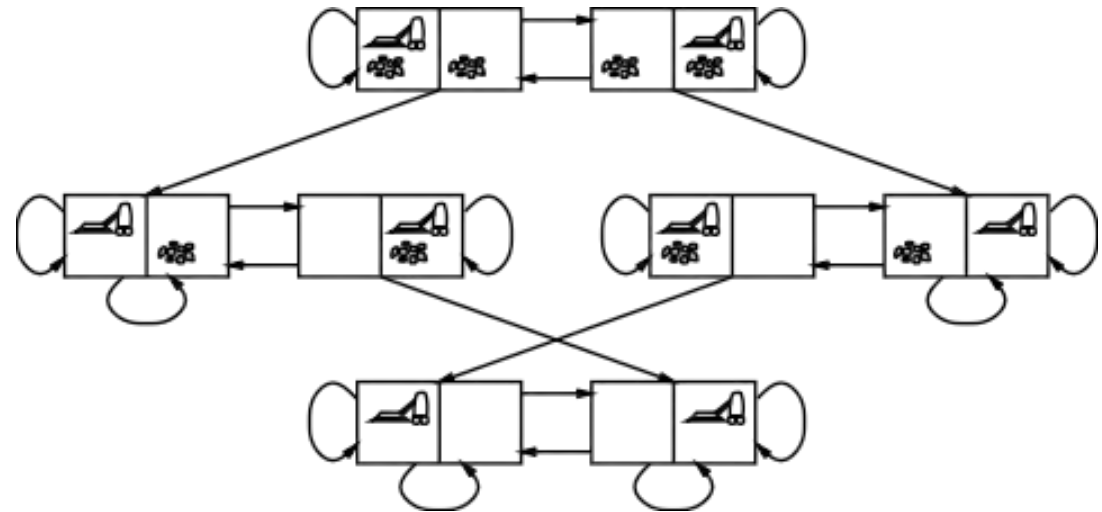
Key Components:

- **Nodes** – represent states in the search space.
- **Edges** – represent actions leading to new states.
- **Start Node** – initial state.
- **Goal Node(s)** – states satisfying the goal test.



Example Vacuum World

- states?: integer dirt and robot locations (ignore dirt amounts etc.)
- actions?: *Left, Right, Suck, NoOp*
- goal test?: no dirt
- path cost?: 1 per action (0 for *NoOp*)



8 tile- Puzzle

- states?: integer locations of tiles (ignore intermediate positions)
- actions?: move blank left, right, up, down (ignore unjamming etc.)
- goal test?: = goal state (given)
- path cost?: 1 per move
- [Note: optimal solution of n -Puzzle family is NP-hard]

7	2	4
5		6
8	3	1

1	2	3
4	5	6
7	8	

Search Strategy Categories

- **Uninformed (Blind) Search**
 - No additional information about states beyond the problem definition.
 - Examples: BFS, DFS, Uniform Cost Search.
- **Informed (Heuristic) Search**
 - Uses domain-specific knowledge (heuristics) to guide search.
 - Examples: Greedy Best-First Search, A*.

- Criteria to evaluate strategies:
 - **Completeness** – Will it find a solution if one exists?
 - **Optimality** – Will it find the best solution?
 - **Time complexity** – Number of nodes generated.
 - **Space complexity** – Maximum nodes stored in memory.

Uninformed Search

Uninformed Search Overview

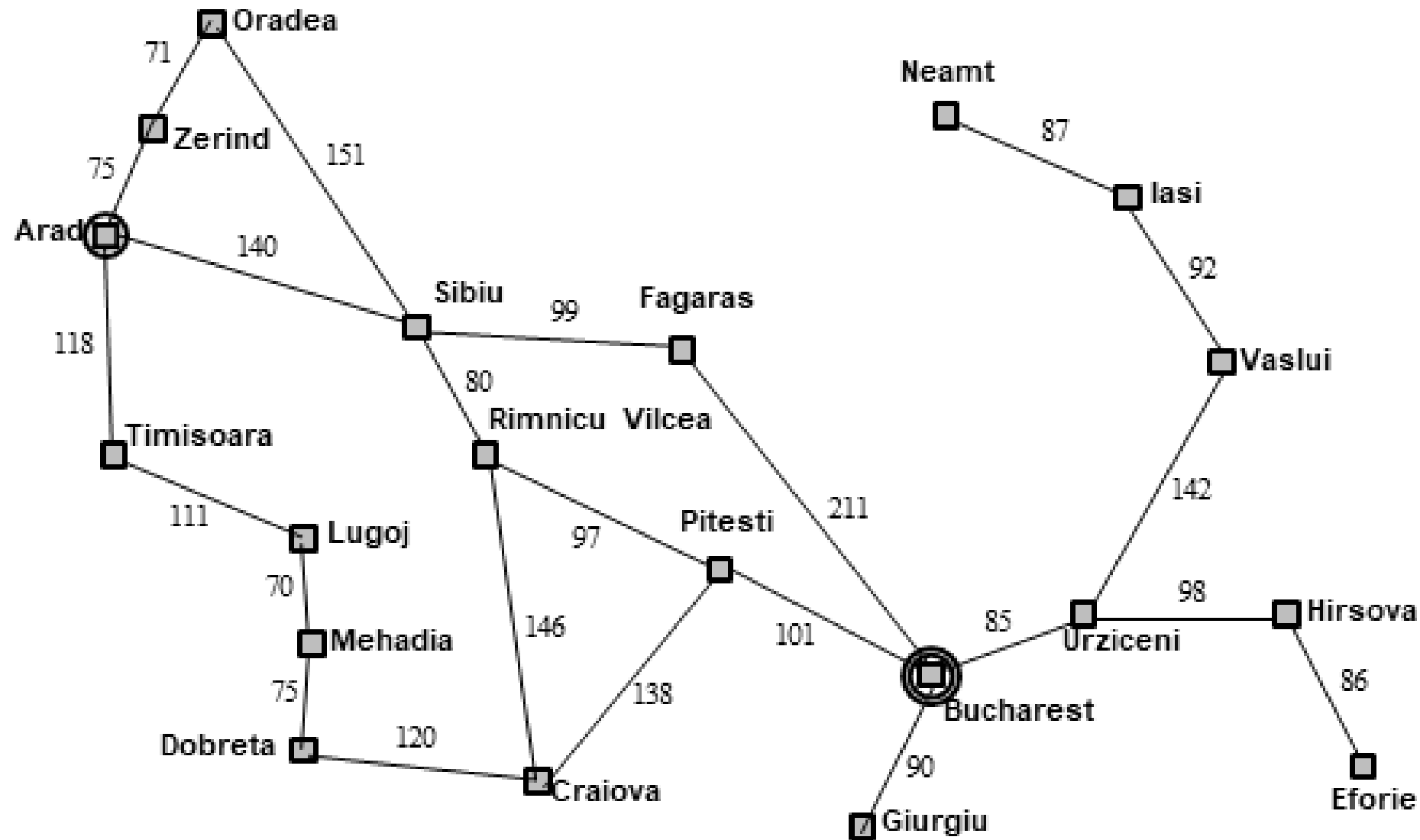
- Definition:
 - Searches without domain-specific knowledge beyond the problem definition.
- Characteristics:
 - Explore the search space systematically.
 - Guarantee completeness if implemented correctly (depending on the method).
- Examples:
 - Breadth-First Search (BFS)
 - Depth-First Search (DFS)
 - Uniform Cost Search (UCS)
 - Depth-Limited Search, Iterative Deepening Search
- When to use:
 - No heuristics available
 - Small or moderate search space

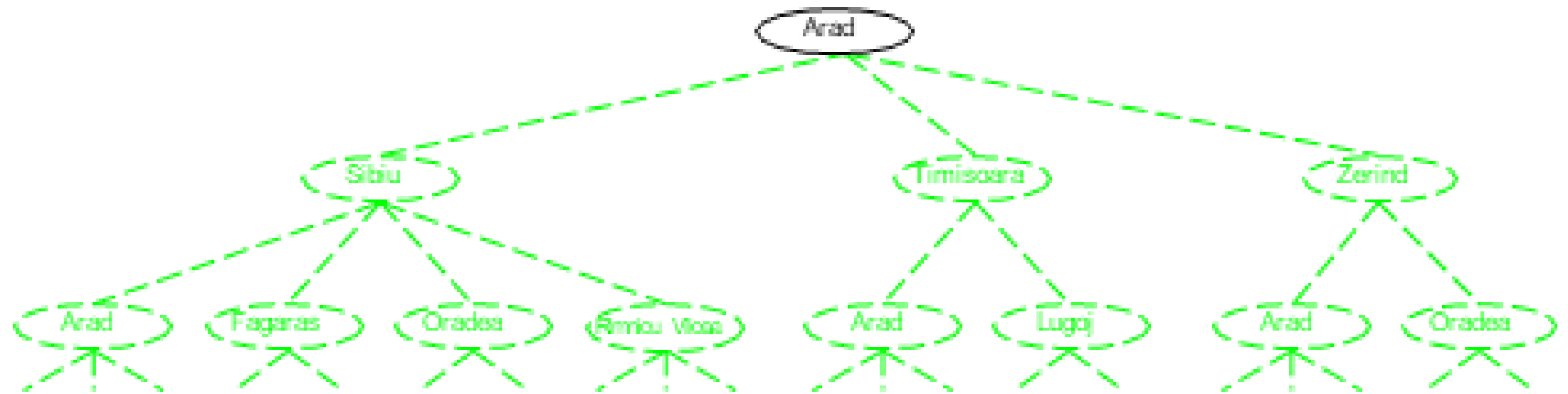
Tree Search Basics

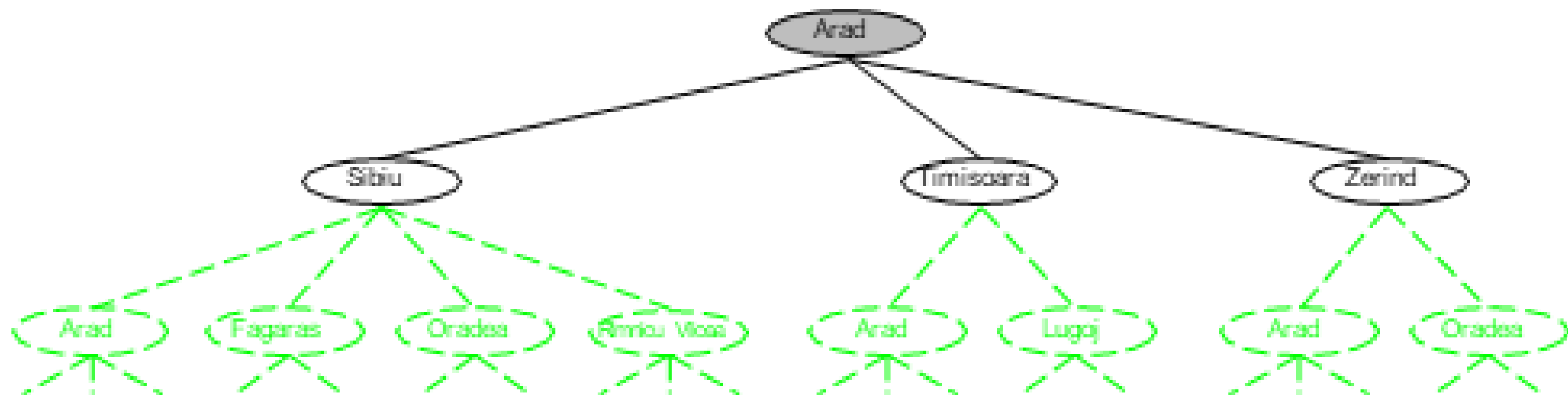
- Basic idea:
 - offline, simulated exploration of state space by generating successors of already-explored states (a.k.a. **expanding** states)

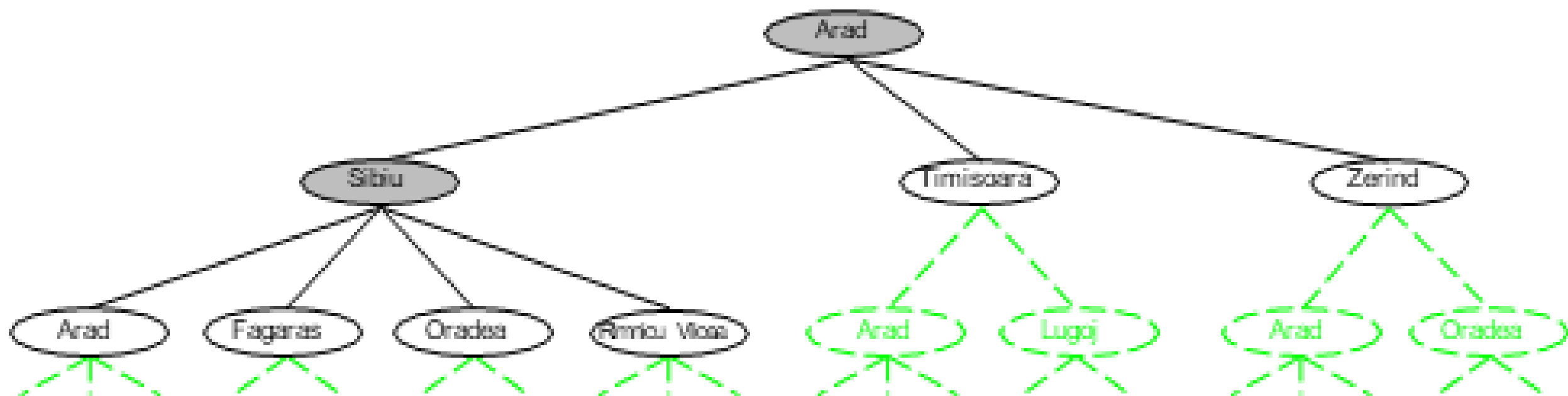
```
function Tree-Search( problem, strategy ) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

Example Problem









General Tree Search

```
function Tree-Search(problem, fringe) returns a solution, or failure
  fringe  $\leftarrow$  Insert(Make-Node(Initial-State[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  Remove-Front(fringe)
    if Goal-Test(problem, State(node)) then return node
    fringe  $\leftarrow$  InsertAll(Expand(node, problem), fringe)
```

```
function Expand(node, problem) returns a set of nodes
  successors  $\leftarrow$  the empty set
  for each action, result in Successor-Fn(problem, State[node]) do
    s  $\leftarrow$  a new Node
    Parent-Node[s]  $\leftarrow$  node; Action[s]  $\leftarrow$  action; State[s]  $\leftarrow$  result
    Path-Cost[s]  $\leftarrow$  Path-Cost[node] + Step-Cost(node, action, s)
    Depth[s]  $\leftarrow$  Depth[node] + 1
    add s to successors
  return successors
```

Tree Search Strategies

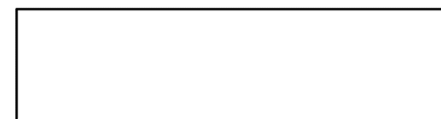
- A strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
- **completeness**—does it always find a solution if one exists?
- **time complexity**—number of nodes generated/expanded
- **space complexity**—maximum number of nodes in memory
- **optimality**—does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b —maximum branching factor of the search tree
 - d —depth of the least-cost solution
 - m —maximum depth of the state space (may be ∞)

Breadth-First Search (BFS)

- Expand the shallowest unexpanded node first.
- **Example:** Delivery robot exploring campus buildings level by level.

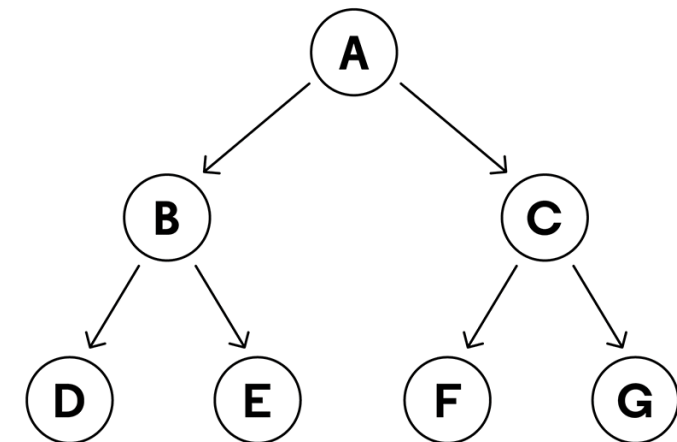
```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure  
  node  $\leftarrow$  NODE(problem.INITIAL)  
  if problem.IS-GOAL(node.STATE) then return node  
  frontier  $\leftarrow$  a FIFO queue, with node as an element  
  reached  $\leftarrow$  {problem.INITIAL}  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if problem.IS-GOAL(s) then return child  
      if s is not in reached then  
        add s to reached  
        add child to frontier  
  return failure
```

```
function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure  
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```



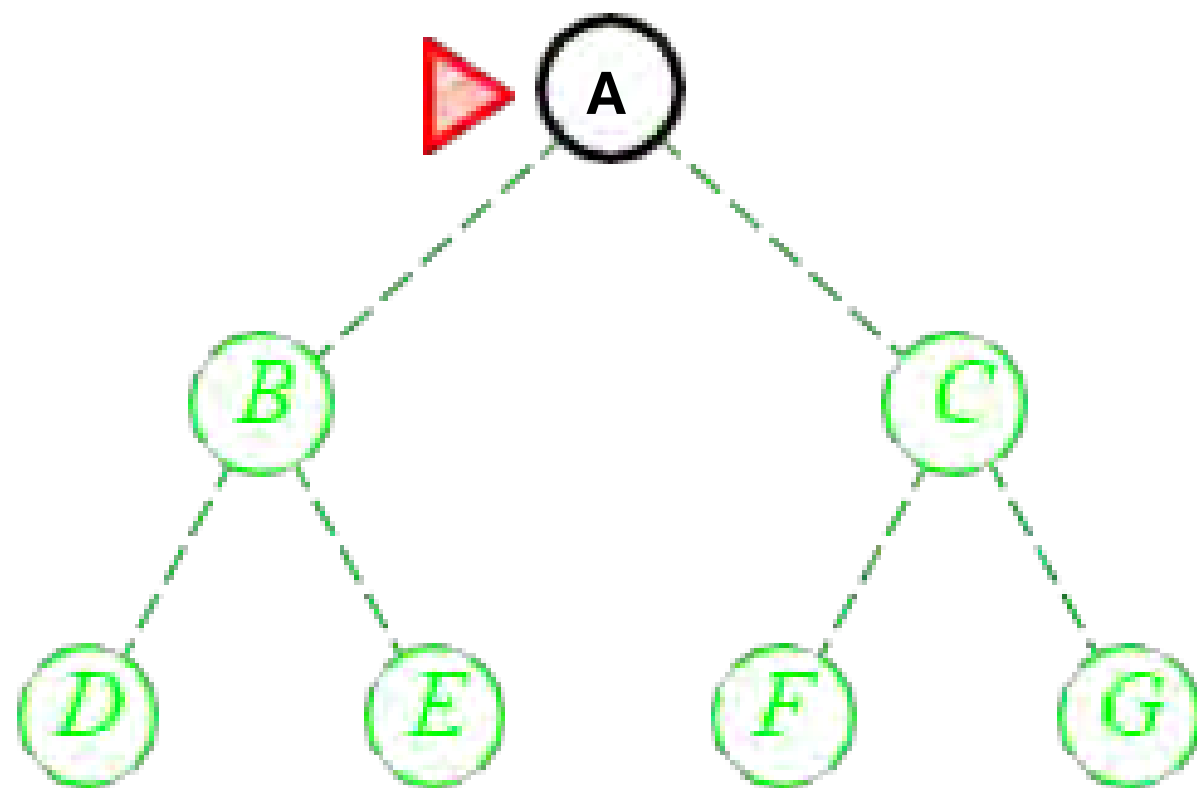
Frontier Queue
FIFO (First in First Out)

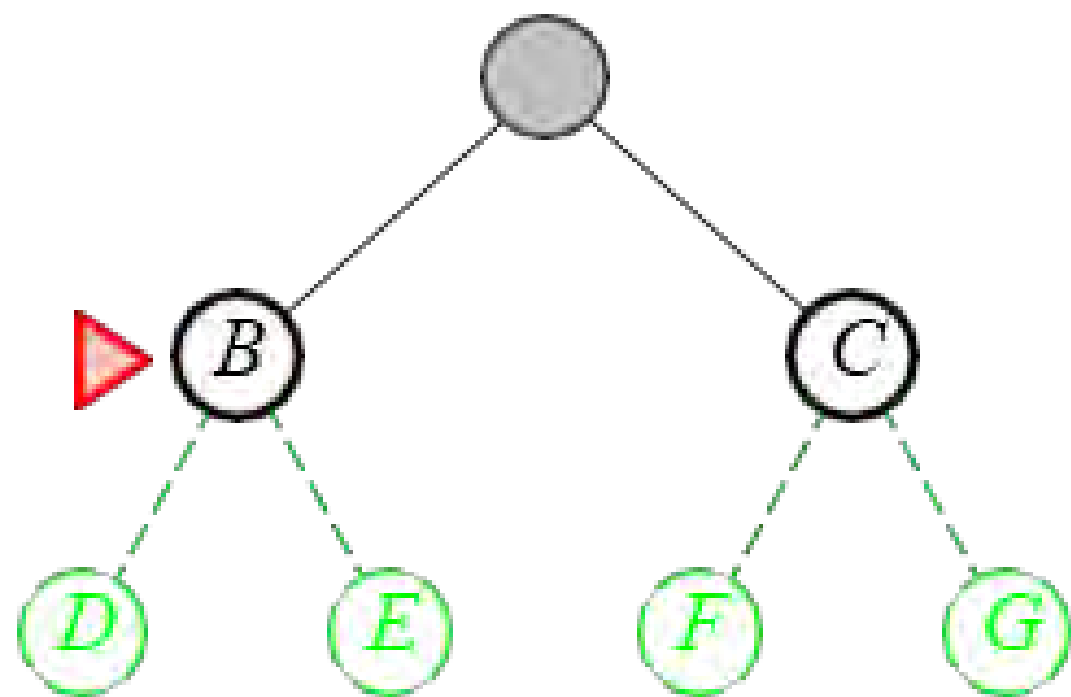
Tree with an Empty Queue

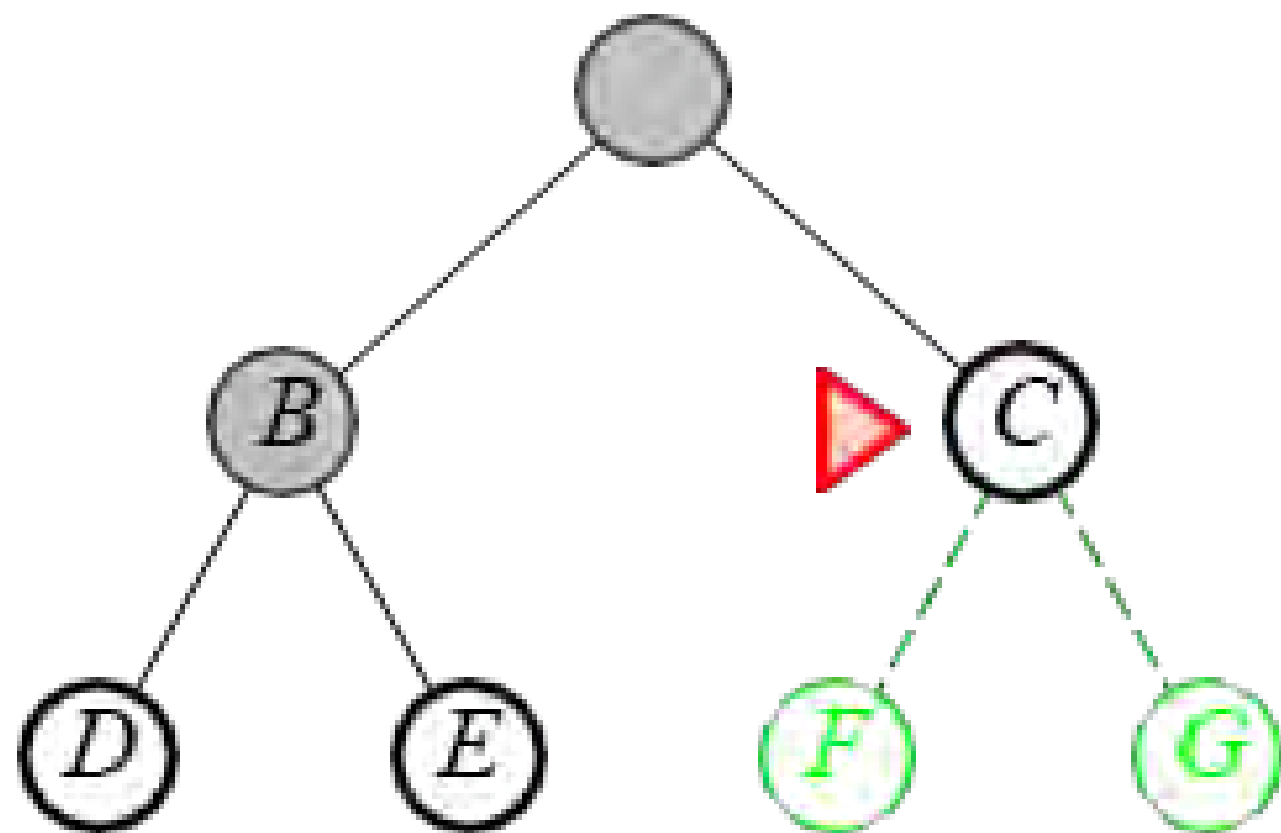


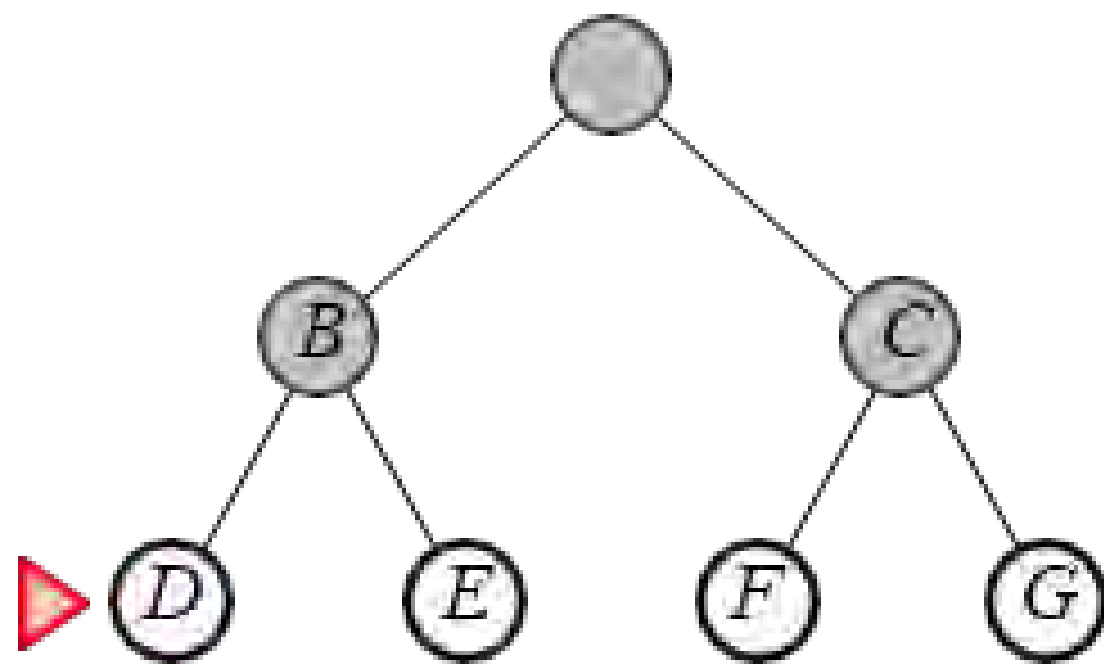
More on BFS

- Complete?? Yes (if b is finite)
- Time?? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d
- Space?? $O(b^{d+1})$ (keeps every node in memory)
- Optimal?? Yes (if cost = 1 per step); not optimal in general
- **Space** is the big problem; can easily generate nodes at 100MB/sec so 24hrs = 8640GB.









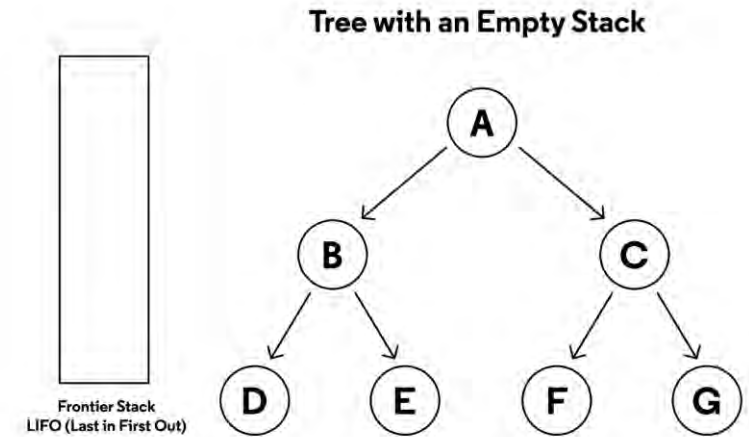
Summary of BFS

Aspect	BFS
Time Complexity	$O(b^d)$ – exponential in the depth d of the shallowest goal
Space Complexity	$O(bd)$ $O(b^d)$ – must store all nodes at the current frontier
When Good	Finding shortest path in terms of number of steps, shallow solutions
When Bad	High branching factor or deep solutions (explodes in memory)

Depth-First Search (DFS)

- **Expand the deepest unexpanded node first.**

- **Example:** Maze navigation without remembering visited cells- can get stuck going deep in wrong path.



Algorithm for DFS

Input: Graph $G(V, E)$

Output: Arrays *pre*, *post* of pre-visit and post-visit times

clock = 1 {Graph traversal counter}

for all $v \in V$ **do**

 visited(v) = false

end for

for all $v \in V$ **do**

if not visited(v) **then**

 explore(v)

end if

end for

Procedure **explore**(v) {Find all nodes reachable from v }

 visited(v) = true

 pre [v] = clock {Mark the time of first discovery of v }

 clock = clock + 1

for each edge $(v, u) \in E$ **do**

if not visited(u) **then**

 explore(u)

end if

end for

 post [v] = clock {Mark the time of final departure from v }

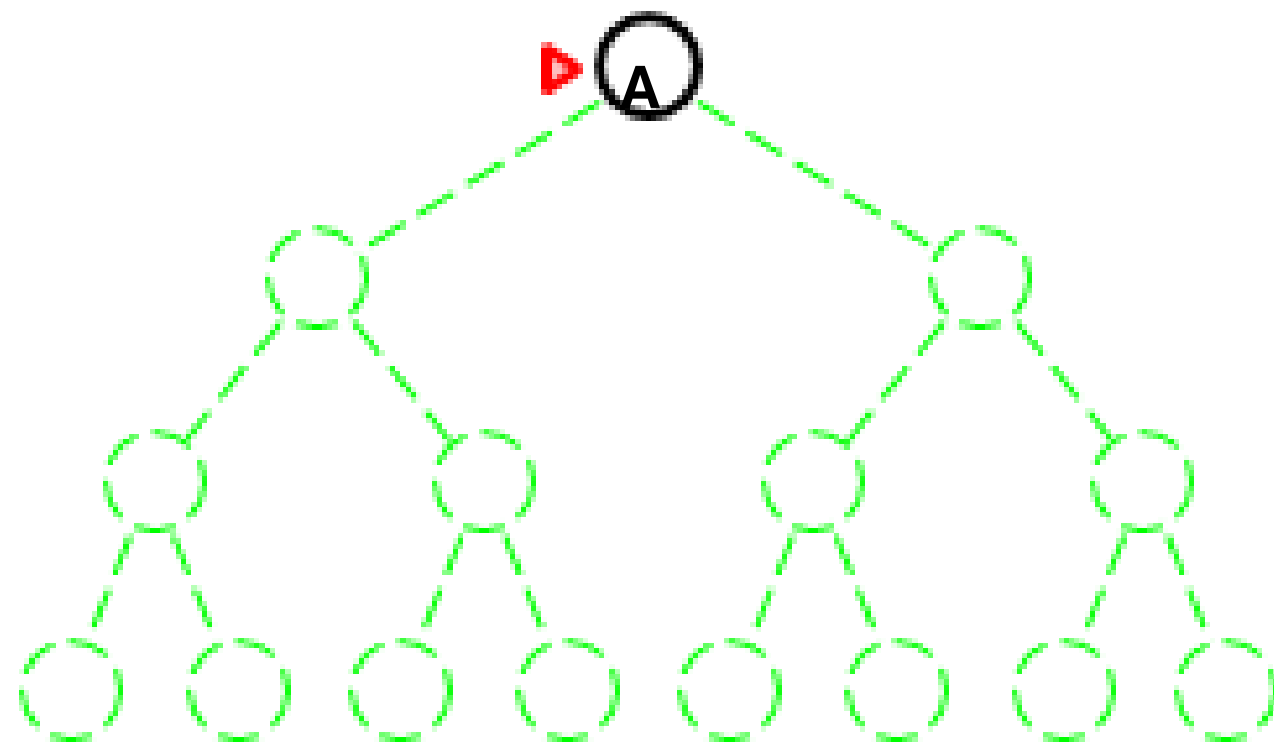
 clock = clock + 1

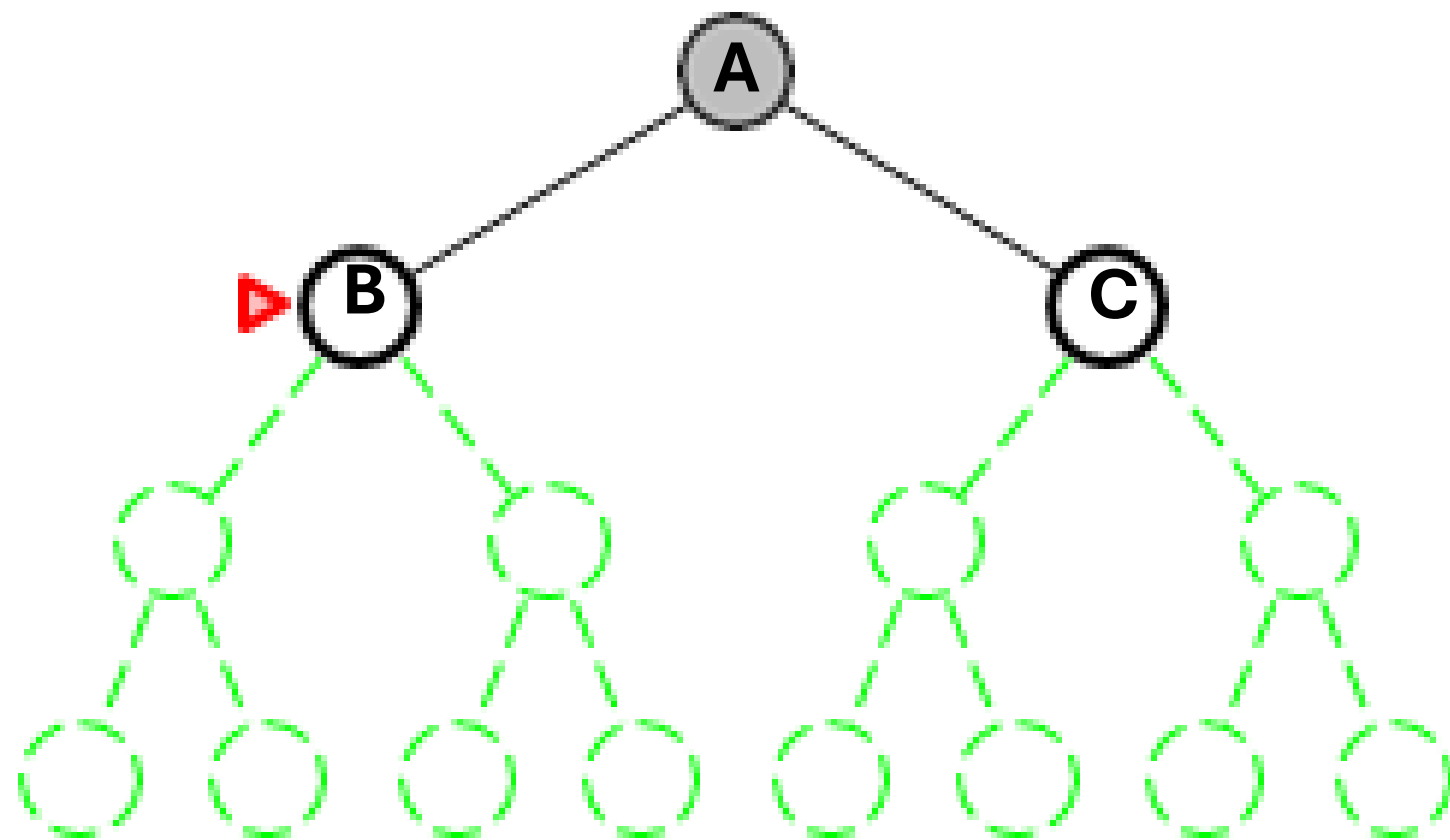
More on DFS

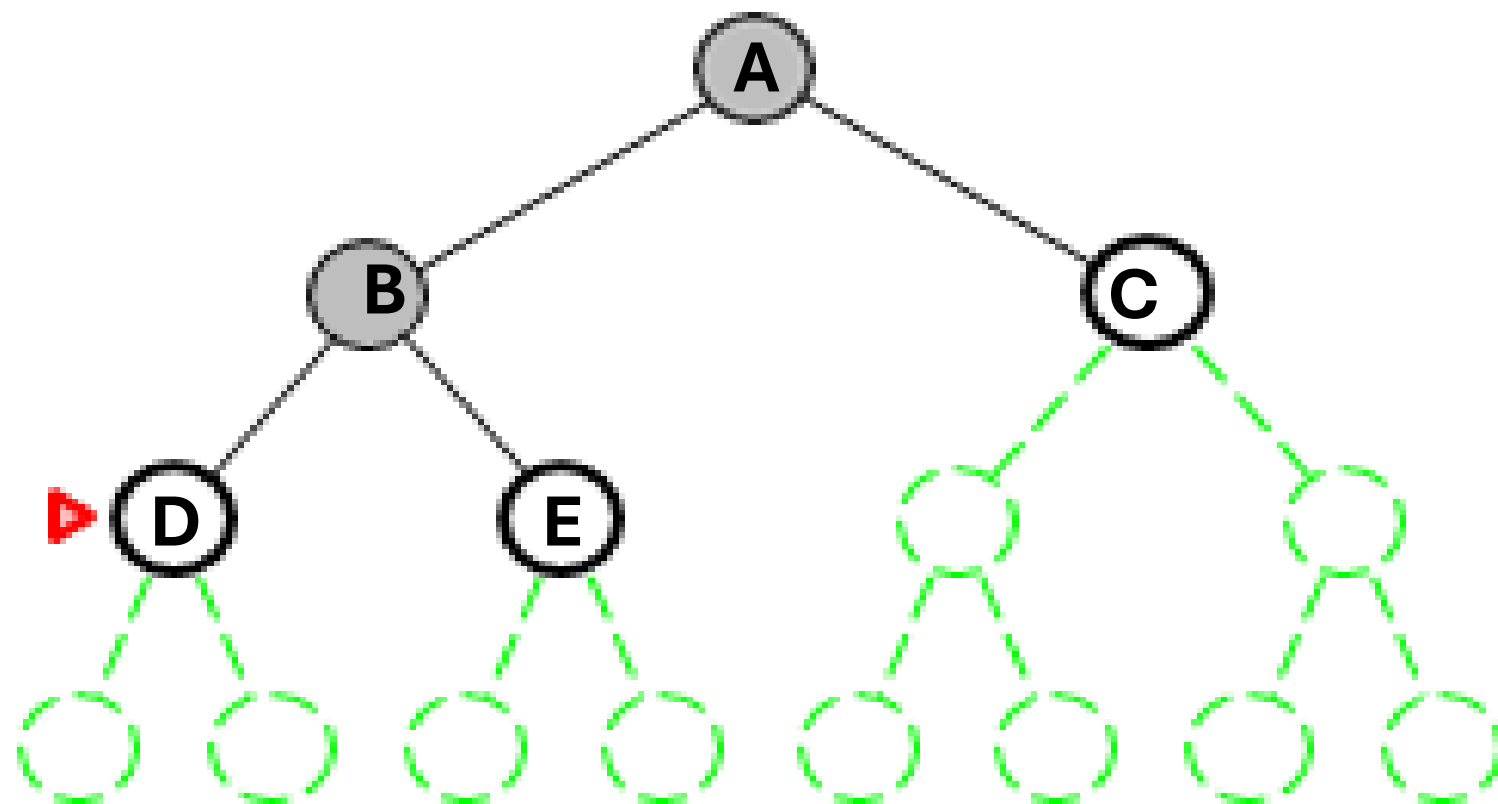
- **Completeness:** complete within finite state space as it will expand every node within a limited search tree.
- **Time Complexity:** Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

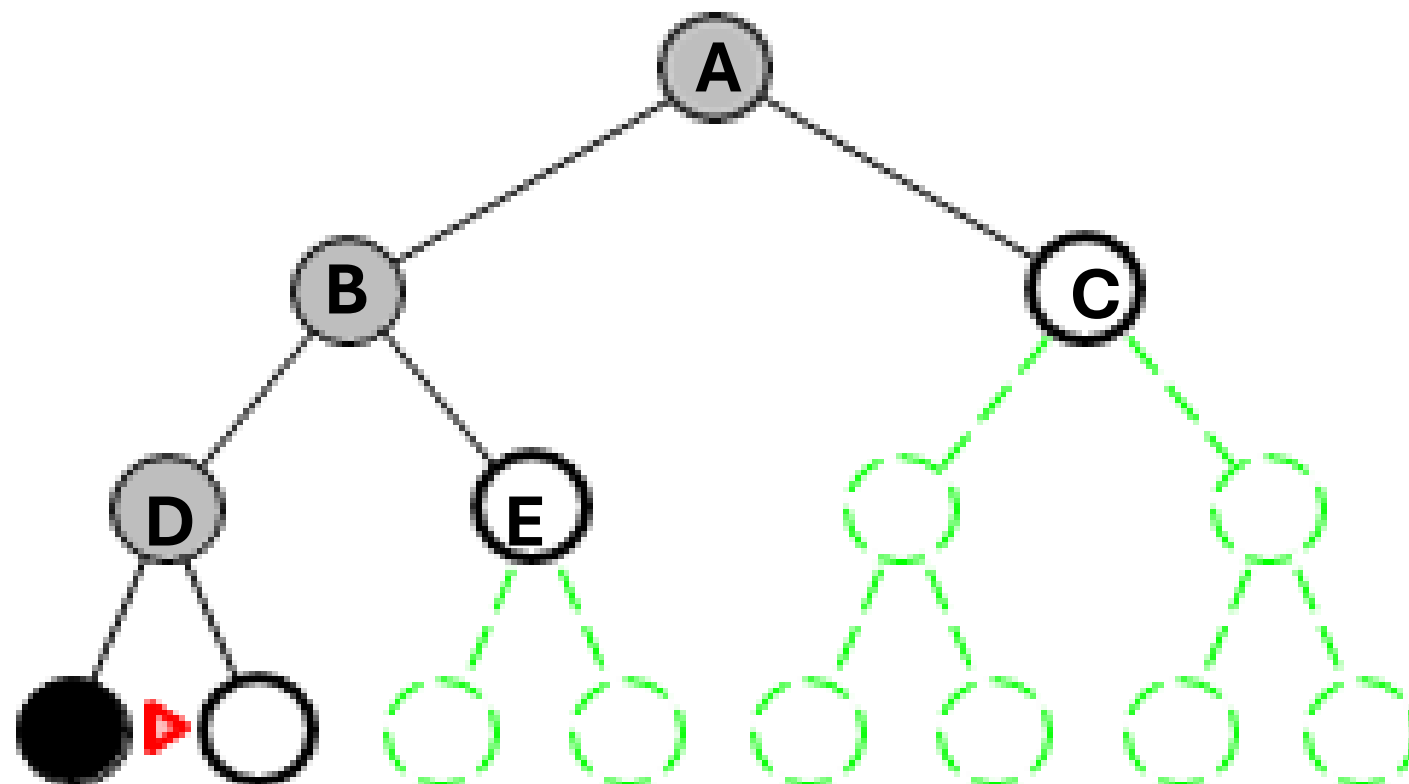
$T(n) = 1 + b^2 + b^3 + \dots + n^m = O(b^m)$, where m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)

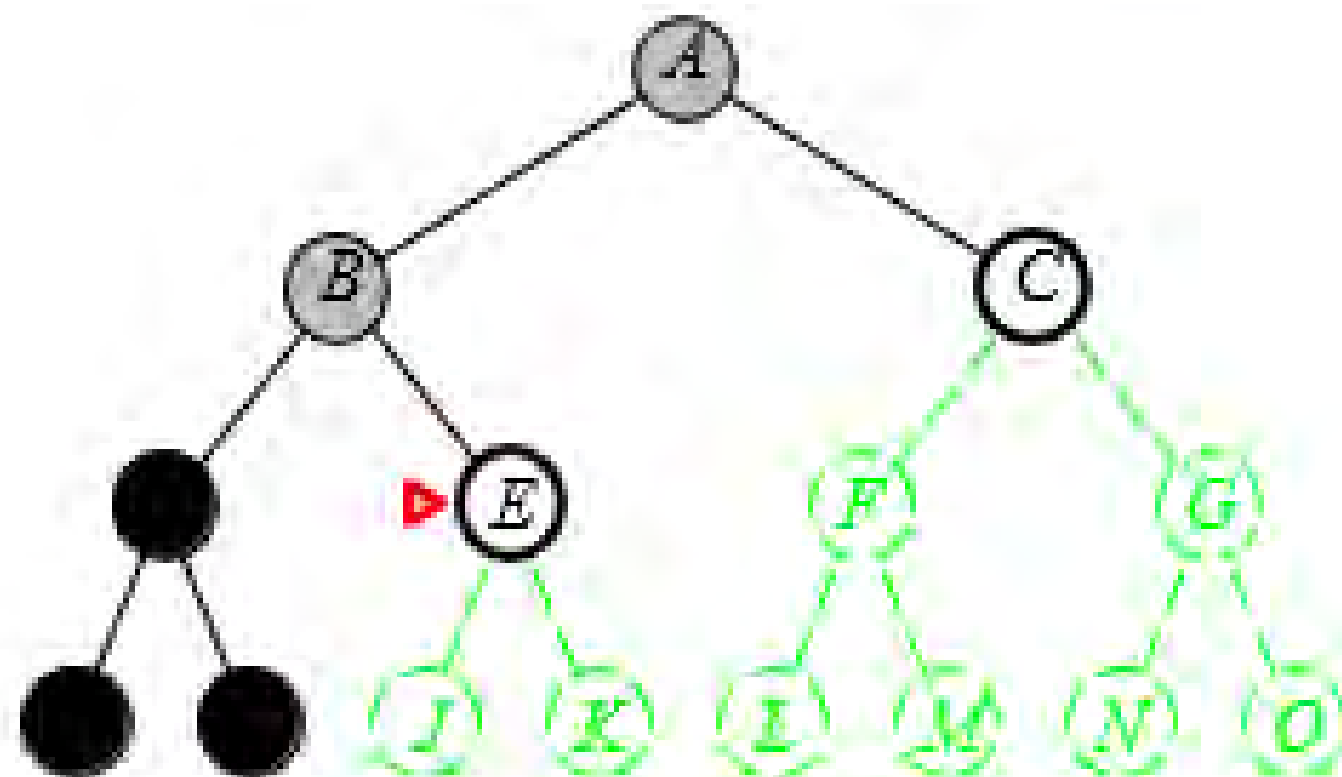
- **Space Complexity:** DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.
- **Optimal:** DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.











Summary of DFS

Aspect	DFS
Time Complexity	$O(b^m)$ – can be very large if m is big
Space Complexity	$O(bm)$ – linear in depth
When Good	Low memory, deep solution exists
When Bad	Infinite or huge depth without early solution

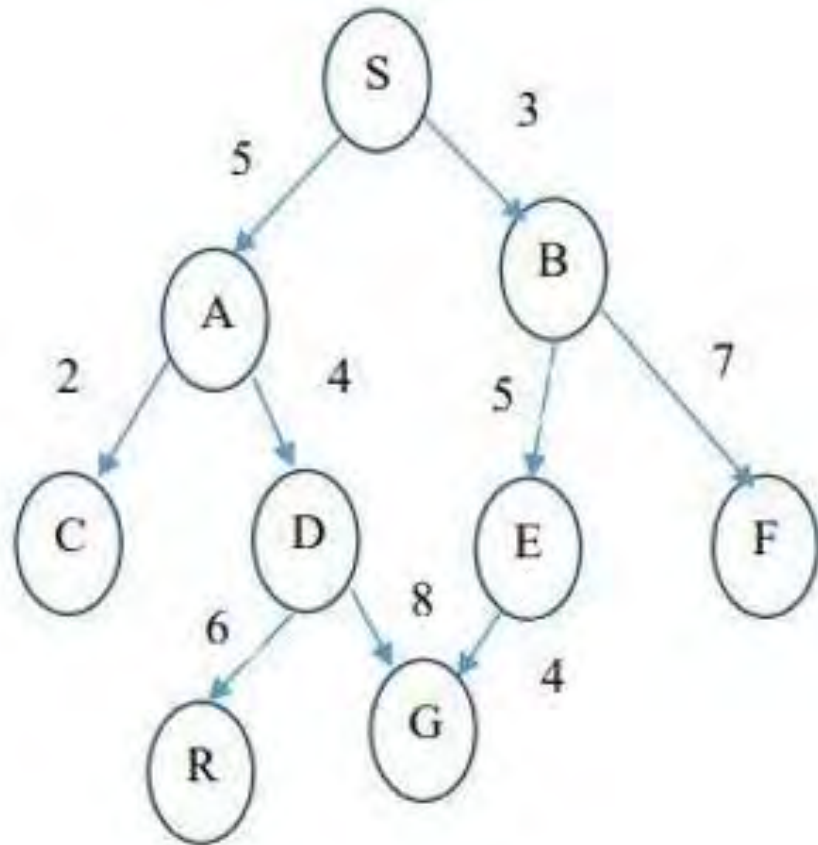
Uniform-Cost Search (UCS)

Expands the **node with the lowest total path cost** $g(n)$ from the start.

Equivalent to **Dijkstra's Algorithm** in graph theory.

More on UCS

- **Implementation:**
 - *fringe* = queue ordered by path cost, lowest first
 - Equivalent to breadth-first if step costs all equal
- Complete?? Yes, if step cost $\geq E$
- Time?? # of nodes with $g \leq$ cost of optimal solution,
 $O(b^{fC_*/d})$
- where C^* is the cost of the optimal solution
- Space?? # of nodes with $g \leq$ cost of optimal solution,
 $O(b^{fC_*/d})$
- Optimal?? Yes—nodes expanded in increasing order of $g(n)$



Activity

- Compute the nodes visited via
 - BFS
 - DFS
 - UCS

Other Search

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes*	Yes*	No	Yes, if $l \geq d$	Yes
Time	b^{d+1}	$b^{fC^*/d}$	b^m	b^l	b^d
Space	b^{d+1}	$b^{fC^*/d}$	bm	bl	bd
Optimal?	Yes*	Yes	No	No	Yes*